Master's degree in Intelligent Systems

Subject: 11754 - Deep Learning

Year: 2025-2026

Professor: Miguel Ángel Calafat Torrens

Introduction

Welcome to the first lab of the Deep Learning course. In this lab you will work with simple neural networks to solve classification problems, both in code and using an interactive web tool.

How to work on the labs: The labs (**all of them, not just this one**) are not just about writing correct code or getting the right output. A key part of the evaluation is your ability to **explain your reasoning**. For every problem, use text cells (markdown) to describe:

- What you tried and why.
- What decisions you made (architecture, hyperparameters) and the reasoning behind them.
- What you observed in the results and what conclusions you draw.

A correct execution without any explanation is considered incomplete. Get used to documenting your thought process — it is a valuable skill both academically and professionally.

So, even if is not said explicitly in the statement, add a text cell with your explanations.

In [1]:
```
# Environment detection: Colab vs local
try:
    import google.colab
    IN_COLAB = True
except ImportError:
    IN_COLAB = False

print(f'Running on {"Google Colab" if IN_COLAB else "local environment"}')
```

```
Running on local environment
```

In [2]:
```
# Setup: Drive mount (Colab) or local path
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/gdrive')
    %cd '/content/gdrive/MyDrive/LABS2026/LAB01'
    %ls -l

# Here the path of the project folder (which is where this file is) is inserted
# into the python path. There's nothing to do; just execute the cell.
import pathlib
import sys
```

```
PROJECT_DIR = str(pathlib.Path().resolve())
sys.path.append(PROJECT_DIR)
```

In [3]:
```
# Importing some libraries
import torch
from torch import nn
from torch import optim
import numpy as np
import matplotlib.pyplot as plt
import helper_L1 as hp
if IN_COLAB:
    from google.colab import files
from IPython.display import clear_output
```

In [4]:
```
# Random seed for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
```

Out[4]:
```
<torch._C.Generator at 0x20d392fc3b0>
```

In [5]:
```
# This is a way to view the content of the help file without having to edit it
# in a new tab. You can also browse to it through the left bar.
if IN_COLAB:
    files.view('helper_L1.py')
```

In [6]:
```
# Define the global variable 'device' to be used where appropriate
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(DEVICE)
```

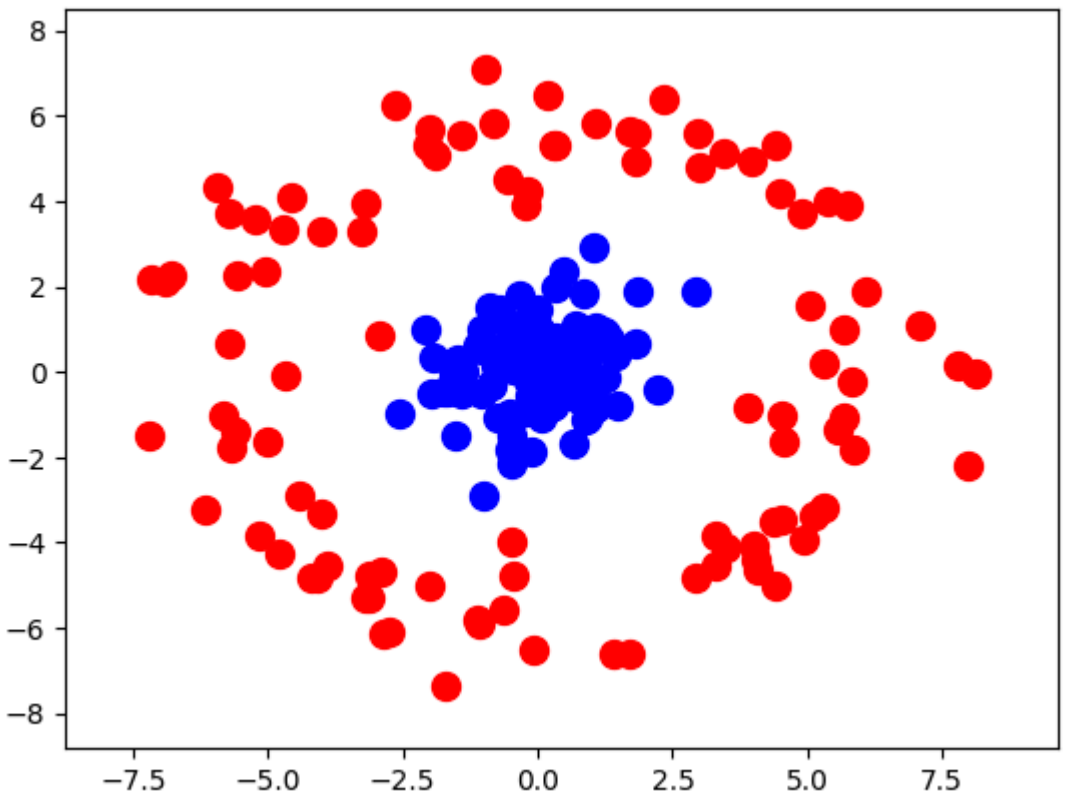
```
cpu
```

Problem 1

In this exercise you are supplied with a cloud of points and it is intended that you will be able to train **the simplest possible model that correctly classifies all the points**.

In this case it is only a question of classifying the training points, so don't worry about overfitting.

In [7]:
```
# Creation of a cloud of dots.

# Generate dots_bck test (background) and dots ground truth
dots_bck, dots_gt = hp.p_gen1(100)

# Create batches of 20 elements
inputs, labels = hp.batch_it(20, dots_gt)

# Visualization of the cloud
hp.my_plot(dots_gt, dots_gt)
```



Training function

A training function is given to you, so you don't have to deal with it. You can just use it as in the example below

In [8]:
```
# Train function
def train(n_epochs, model, optimizer, criterion, inputs, labels, dots_tst,
          dots_gt, each=2):
    """
    A training function with graphical info every "each" epochs.

    You don't have to tweak anything here. Just take into account the number
    of epochs and the "each" parameter when you call this function.
    """
    acum_loss = []
    model = model.to(DEVICE)
    for epoch in range(n_epochs):
        batch_loss = 0.0
        for Xb, Yb in zip(inputs, labels):
            Xb, Yb = Xb.to(DEVICE), Yb.to(DEVICE)
            # Training algorithm

            # Reset the gradients
            optimizer.zero_grad()

            # Forward pass
            Yp = model.forward(Xb)

            # Loss
            loss = criterion(Yp, Yb)

            # Backpropagation
            loss.backward()
```

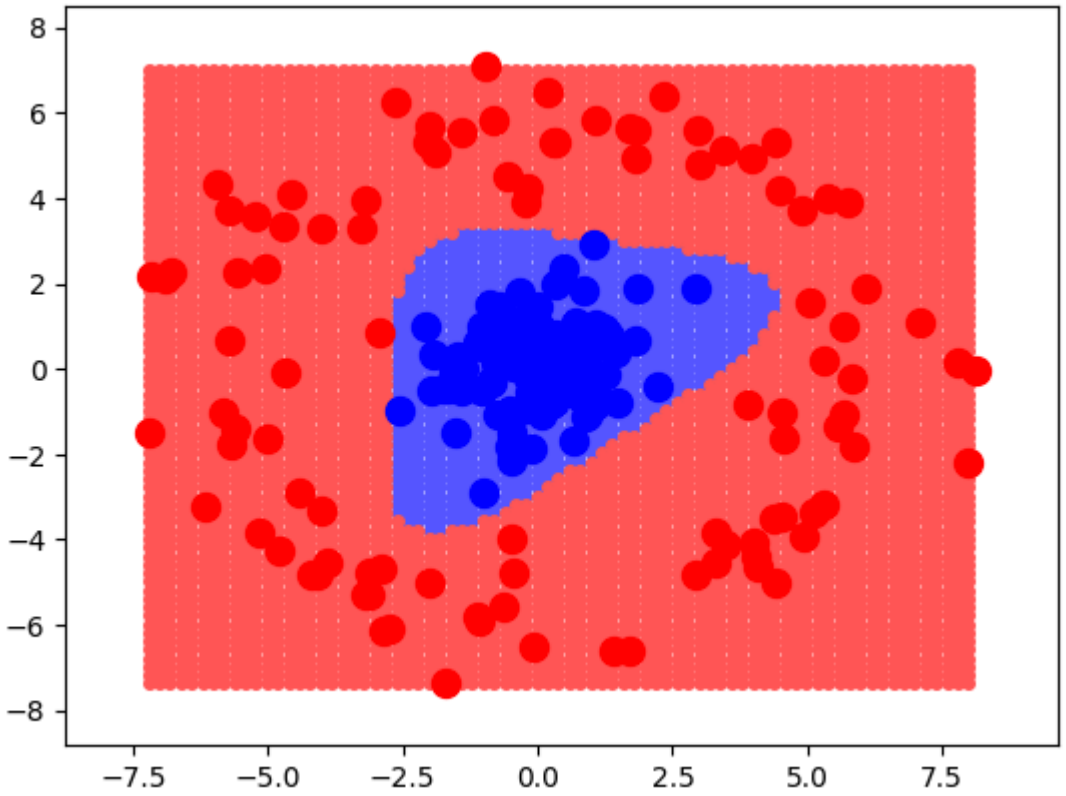
```
            # Update weights
            optimizer.step()

            batch_loss += loss.item()

        # Loss log per epoch
        acum_loss.append(batch_loss)

        # Show results every "each" epochs.
        if epoch % each == 0:
            print('Epoch {}: loss = {}'.format(epoch, batch_loss))

            # The grid of points that make up the background is collected
            t_test_points = torch.from_numpy(dots_tst[:2, :].T).to(DEVICE)

            # The outputs of the model are calculated, both for the points that
            # make up the background and for the cloud of points to be
            # classified
            with torch.no_grad():
                model.eval()
                y_test = model.forward(t_test_points)
                y_tr_predict = (model.forward(inputs.reshape(-1, 2).to(
                    DEVICE)) >= 0.5).float()

            # Accuracy is calculated
            hp.acc(y_tr_predict, labels)

            # Transform tensors to np-arrays to be used in my_plot() function
            y_test = y_test.cpu().numpy().round().T
            test_block = np.concatenate((dots_tst[:2, :], y_test), axis=0)
            clear_output(wait=True)
            hp.my_plot(test_block, dots_gt)

            model.train()

    # Accuracy
    with torch.no_grad():
        model.eval()
        y_tr_predict = (model.forward(inputs.reshape(-1, 2).to(
            DEVICE)) >= 0.5).float()

    # Accuracy is calculated
    hp.acc(y_tr_predict, labels)

    return model, acum_loss
```

Set up for training

At this point some start configuration is given to you, just as an example; but **you will need to change it (at least the model)** to reach the expected results.

In [9]:
```
class Network_01(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(in_features=2, out_features=4, bias=True)
        self.tanh = nn.Tanh()
        self.fc2 = nn.Linear(in_features=4, out_features=1, bias=True)
```

```
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.tanh(self.fc1(x))
        x = self.sigmoid(self.fc2(x))
        return x

np.random.seed(SEED)
torch.manual_seed(SEED)

# Instanciate the model
model = Network_01()

# Instanciate the optimizer with the SGD algorithm (Stochastic Gradient Descent)
optimizer = optim.SGD(model.parameters(), lr=0.1)

# Instanciate the loss function
criterion = nn.MSELoss()

# Simulation
model, _ = train(400, model, optimizer, criterion, inputs, labels, dots_bck,
          dots_gt, 20)
```




```
Acc: 100.0000
```

First of all visit this link

In the link above you will find a demo where you can define your custom neural network trying to fit it to different datasets. Play with it with the most similar dataset (the first one), to check what's your simplest network that can fit correctly.

Once you decide your network, put it here (substitute the perceptron network given to you by your choice) and run the simulation again.

Feel free to tinker all the hiperparameters you need.

Hint: In the web simulation select:

- No regularization
- Problem type: Classification
- Discretize output: Enabled

Your comments:

I approached this task as a model-capacity search problem: start with the simplest hypothesis class and increase complexity only when the decision boundary cannot be represented.

1. **Baseline (linear model):** a single perceptron (2 -> 1, sigmoid) fails systematically because this dataset is not linearly separable (inner cluster vs outer ring). The error pattern is structural, not just optimization noise.
2. **Controlled capacity increase:** I introduced one hidden layer with tanh and tested small widths (2, 3, 4 neurons). Width 2-3 improved fit but left persistent local pockets of error across seeds/initializations.
3. **Selected architecture:** 2 -> 4 -> 1 with tanh in hidden layer and sigmoid output. This is the first architecture that reached a clean 100% training classification in a stable way.

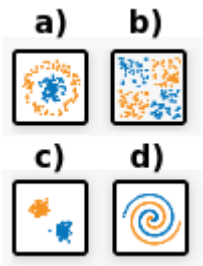
Final setup:

- Architecture: 2 -> 4 -> 1
- Activations: tanh (hidden), sigmoid (output)
- Optimizer: SGD, lr = 0.1
- Loss: MSE
- Epochs: 400

Interpretation: the hidden layer learns a non-linear embedding where a near-linear separation becomes possible at output level. The chosen model is minimal in depth and near-minimal in width for perfect fit, so it satisfies the simplicity criterion while still capturing the topology of the data.

Problem 2

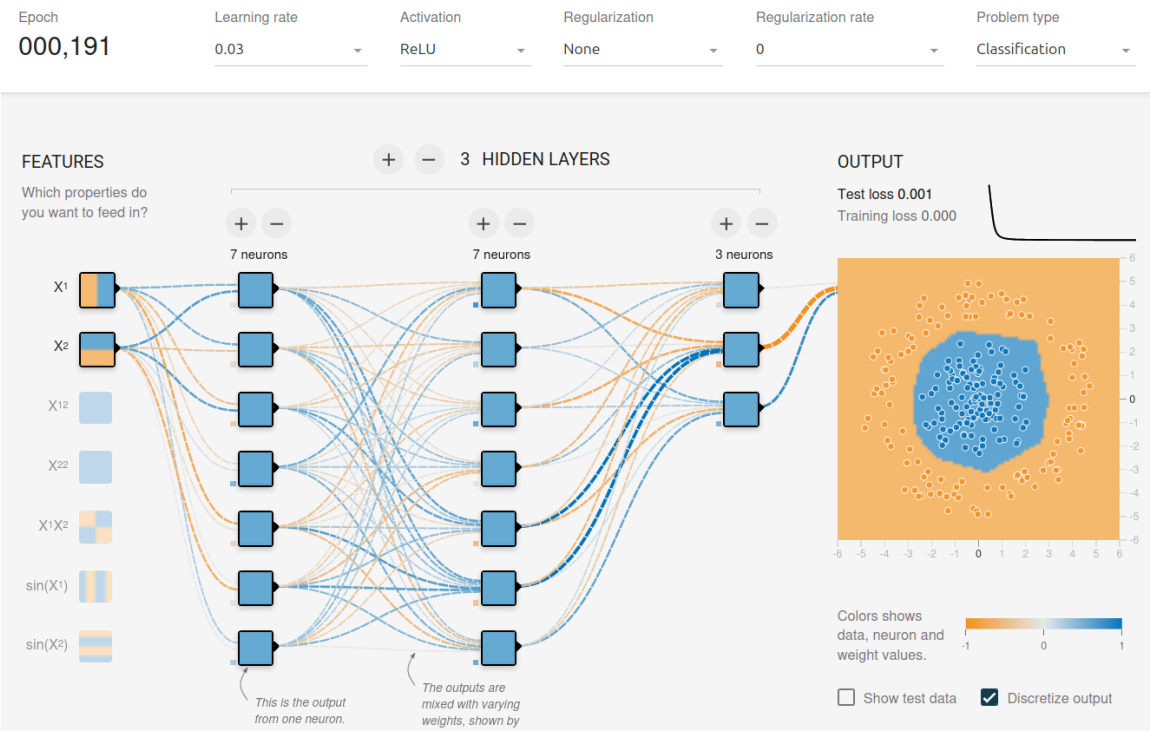
Repeat in the web the problem above with the rest of datasets ('*b*', '*c*' and '*d*' in the image below). You don't have to repeat the exercise with dataset '*a*'.



You just have to inform here the simplest model that correctly classifies all the points.

Note that you don't have to solve the problem in the notebook, since the datasets aren't given to you. Everything has to be done in the web. **Here you have to show the results copying and pasting a snapshot**. In addition to the snapshot, you must write a brief explanation of your choice: what architecture did you select, why do you think it is the simplest one that solves the problem, and what alternatives did you consider.

For example, imagine that you found the best solution in the dataset '*a*' to be 3 hidden layers with 7, 7 and 3 units each (it's just an example that is obviously too complex). Then you should put here a snapshot where the configuration and all the hyperparameters can be seen, just like this one below, and also a text cell with your explanations:



Obviously, you could need a different model for each different dataset.

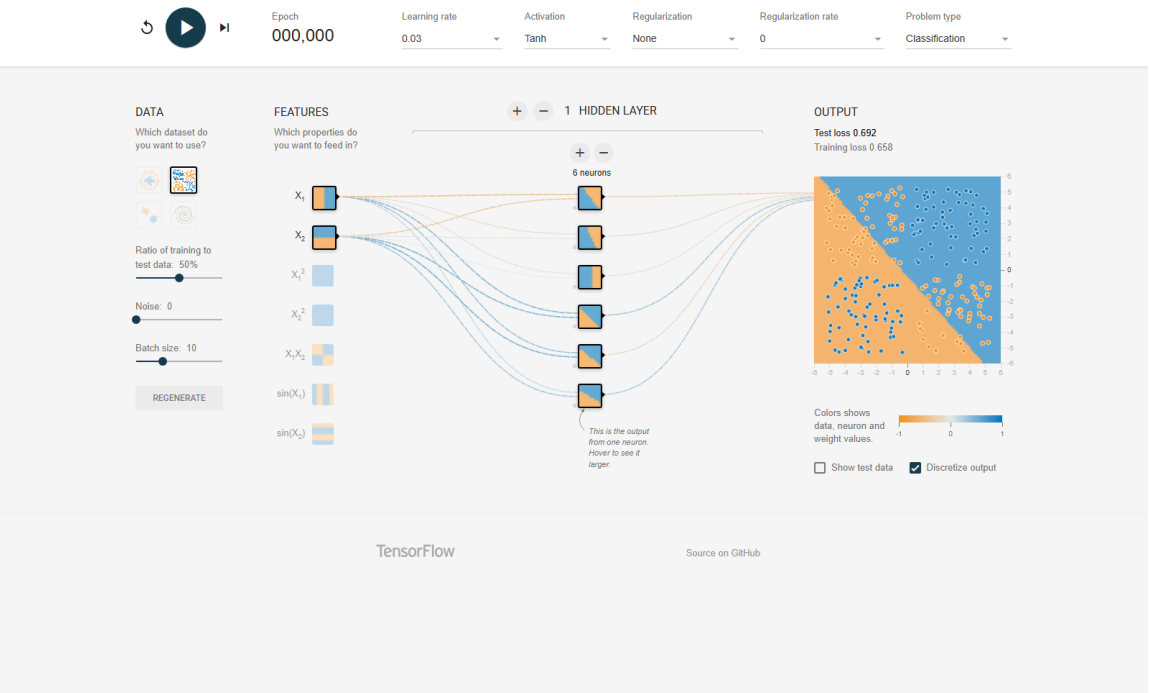
Answer:

I solved datasets **b, c and d** with the same methodology: incremental search from low-capacity models to the first stable architecture that classifies all points correctly (with no regularization and discretized output enabled).

For each case below, I include:

- the final architecture and hyperparameters,
- alternatives I tested and why they were rejected,
- an interpretation of what the chosen model is learning from the geometry of the dataset.

Dataset b)



For **dataset b (XOR-like structure)** I selected a compact but clearly non-linear model.

Chosen model (Playground):

- Hidden layers: 1
- Units: 6
- Activation: tanh
- Learning rate: 0.03
- Regularization: none

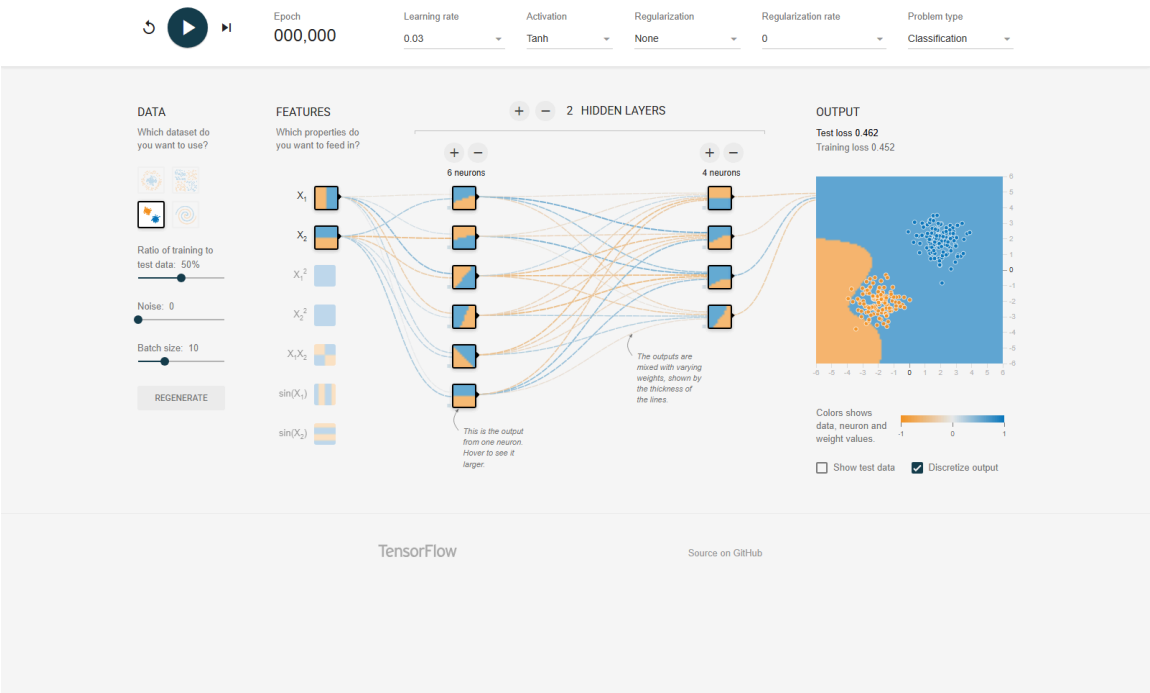
What I tested before:

- 1 hidden layer with 3 or 4 units: underfitting at quadrant transitions; residual misclassified islands remained.

Interpretation:

- XOR needs multiple local linear cuts combined non-linearly; 6 tanh units provide enough basis functions to carve alternating regions without adding unnecessary depth.
- This configuration is the smallest one that was stable in repeated resets, so it balances simplicity and robustness (not just a lucky run).

Dataset c)



Dataset c requires a smoother but more compositional boundary than dataset b.

Chosen model (Playground):

- Hidden layers: 2
- Units: 6, 4
- Activation: tanh
- Learning rate: 0.03
- Regularization: none

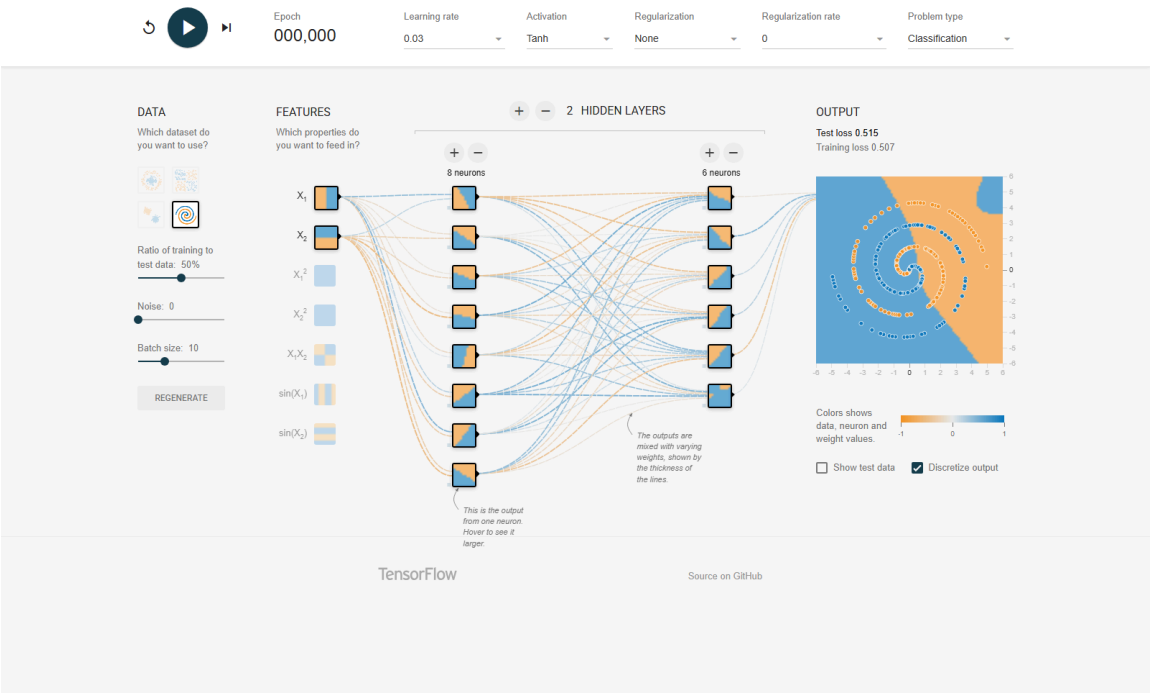
What I tested before:

- Single-hidden-layer variants: consistently underfit curved transition zones between the two groups.

Interpretation:

- The first hidden layer builds intermediate non-linear features; the second layer recombines them into a cleaner global contour.
- Compared with wider 1-layer models, this 2-layer 6-4 architecture gave better boundary smoothness with fewer spurious local flips, i.e., better structural fit with moderate capacity.

Dataset d)



Dataset d (spiral) is the most topologically complex case and demands higher expressive power.

Chosen model (Playground):

- Hidden layers: 2
- Units: 8, 6
- Activation: tanh
- Learning rate: 0.03
- Regularization: none

What I tested before:

- 2-layer 6-4 (and nearby smaller variants): captured part of the spiral but failed in high-curvature turns, leaving systematic local errors.

Interpretation:

- Spiral separation requires representing repeated angular/radial alternations; this needs more local non-linear partitions than the previous datasets.
- The 8-6 network is the first configuration that tracks the winding structure end-to-end without obvious blind spots, so it is the smallest reliable model for full classification here.

Problem 3

Repeat the steps of problem 1 with the next cloud of points. In this case **you don't have to correctly classify all the points**; it is simply a matter of finding a network that you think classifies well enough without overfitting. That is to say, we are trying to find a model that generalizes well.

Comments:

Comments (Problem 3)

I treated this section as a **bias-variance tradeoff** exercise rather than a pure accuracy maximization task.

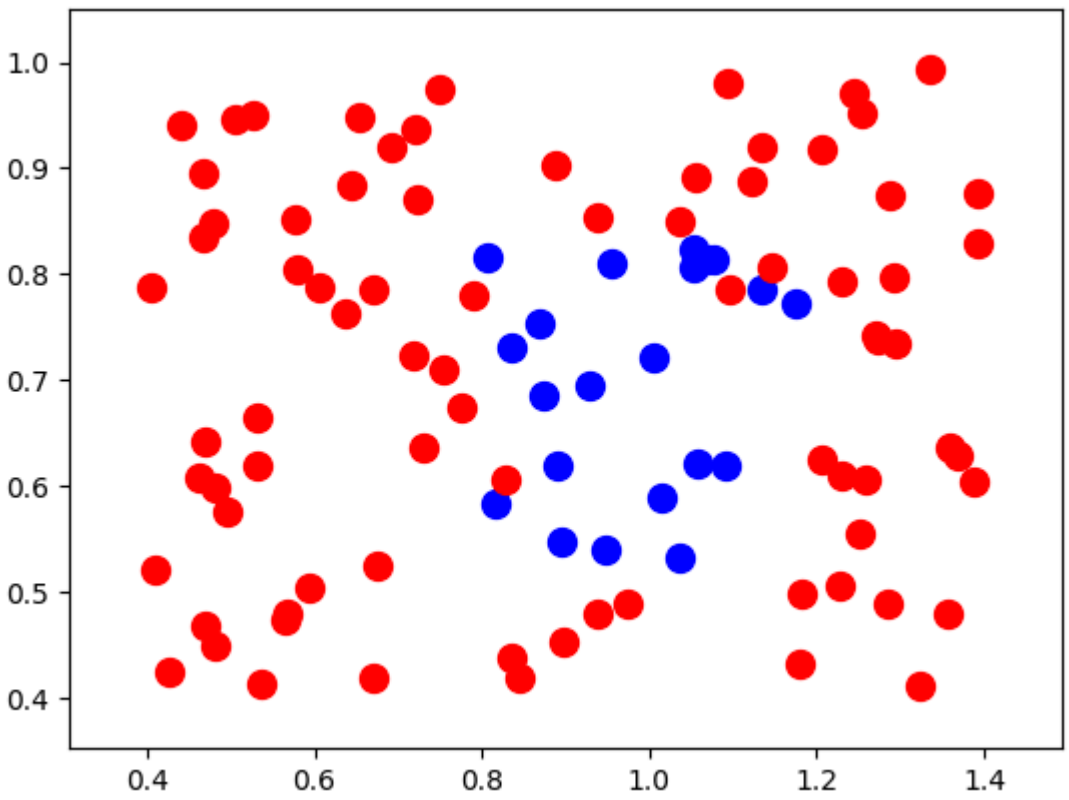
Model used: 2 -> 12 -> 8 -> 1 (ReLU hidden, sigmoid output), Adam, lr = 0.01, BCE, 1500 epochs.

Interpretation and justification:

- The boundary captures the dominant shape of the blue region while avoiding very sharp micro-boundaries around isolated points.
- Larger networks improved training fit but started to create fragmented contours (a classic overfitting sign in this synthetic setting).
- Smaller networks were smoother but missed relevant curvature (high bias).

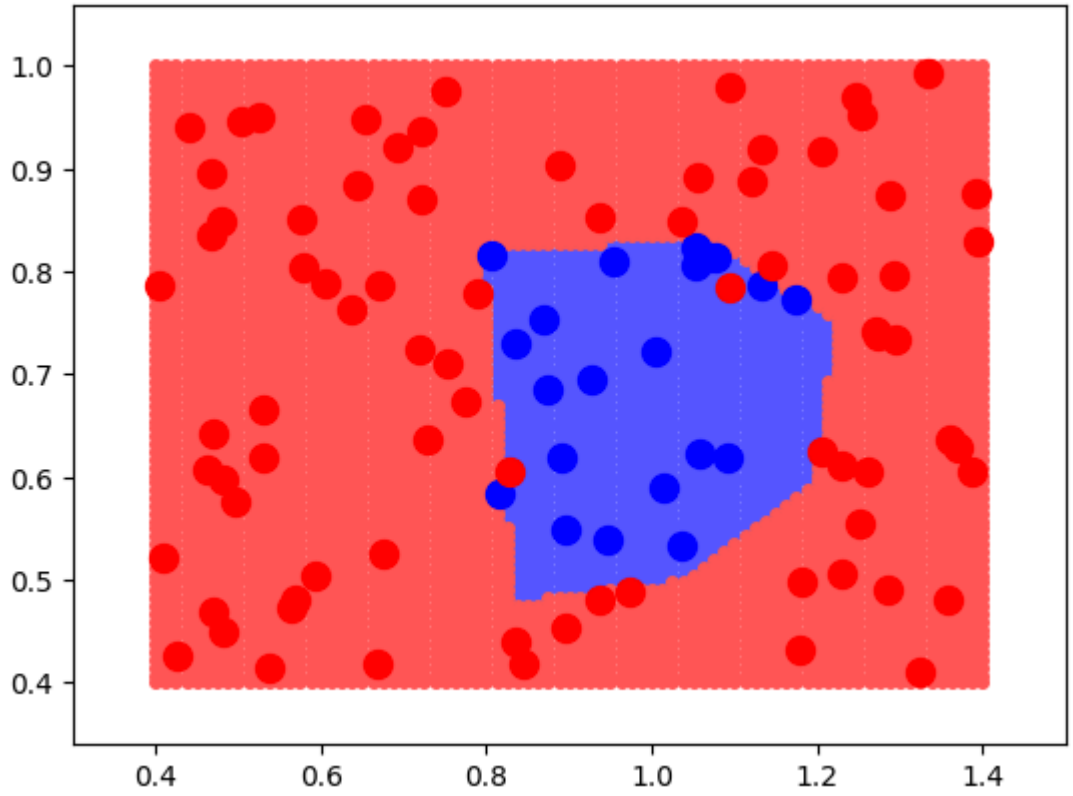
So this configuration is a practical midpoint: enough representational power to model the main structure, but restrained enough to preserve generalization behavior. Final training accuracy in my run: **97%**.

In [10]:
```
# Creation of a cloud of dots.

# Generate dots_bck test (background) and dots ground truth
dots_bck, dots_gt = hp.p_gen2(100)

# Create batches of 20 elements
inputs, labels = hp.batch_it(20, dots_gt)

# Visualization of the cloud
hp.my_plot(dots_gt, dots_gt)
```



In [11]:
```
class Network_02(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(in_features=2, out_features=12, bias=True)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(in_features=12, out_features=8, bias=True)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(in_features=8, out_features=1, bias=True)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu1(self.fc1(x))
        x = self.relu2(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x


np.random.seed(SEED)
torch.manual_seed(SEED)

# Instanciate the model
model = Network_02()

# Instanciate the optimizer
lr = 0.01
optimizer = optim.Adam(model.parameters(), lr=lr)

# Instanciate the loss function
criterion = nn.BCELoss()

# Simulation
n_epochs = 1500
each = 150
```

```
model, _ = train(n_epochs, model, optimizer, criterion, inputs, labels, dots_bck,
          dots_gt, each)
```




```
Acc: 97.0000
```

Problem 4:

Repeat the problem with the next dataset. In this case **I want you to insanely overfit the model** so it classifies correctly all the points. But don't forget to **select the simplest model** that can do the job.

Comments:

Comments (Problem 4)

Here the objective is intentionally different: prioritize perfect training fit, then minimize architecture size under that constraint.

Selected model: 2 -> 10 -> 10 -> 1 (tanh, tanh, sigmoid), Adam, lr = 0.01, BCE, 6000 epochs.

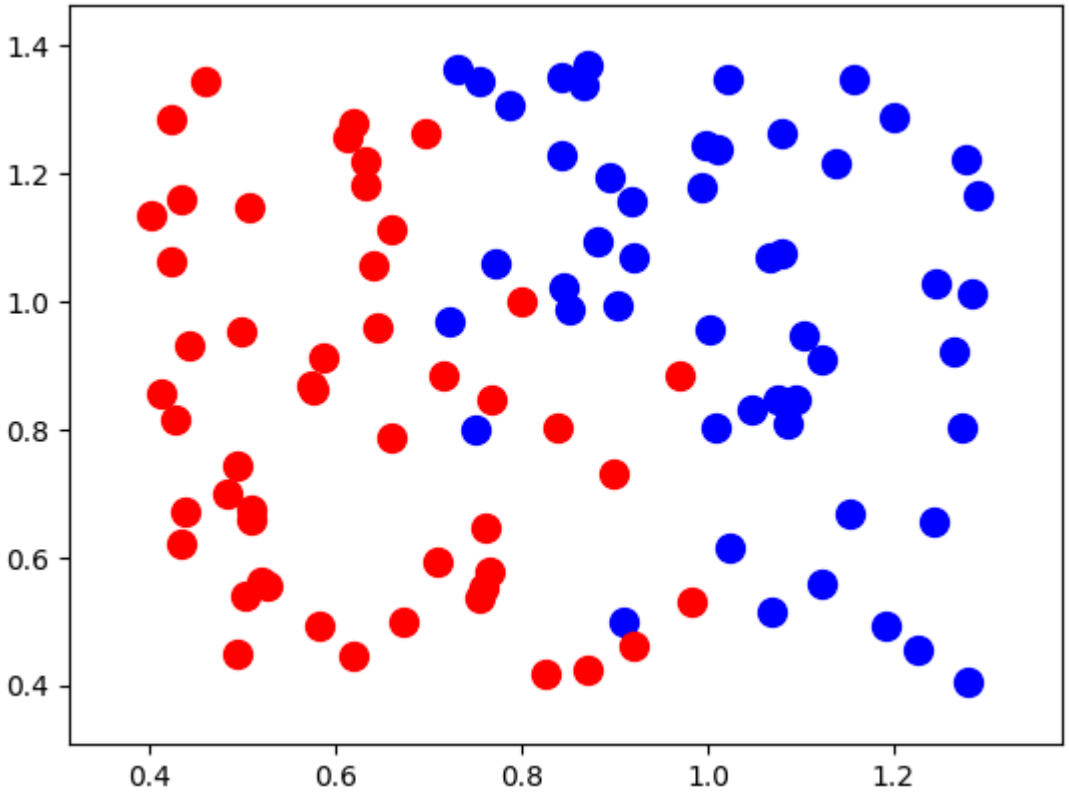
Why this is the best compromise for the stated goal:

- Narrower alternatives did not consistently reach full point-wise classification.
- Wider/deeper alternatives also reached 100%, but added unnecessary complexity.
- This 10-10 setting is therefore the smallest stable overfitting architecture I found for this dataset.

Interpretation:

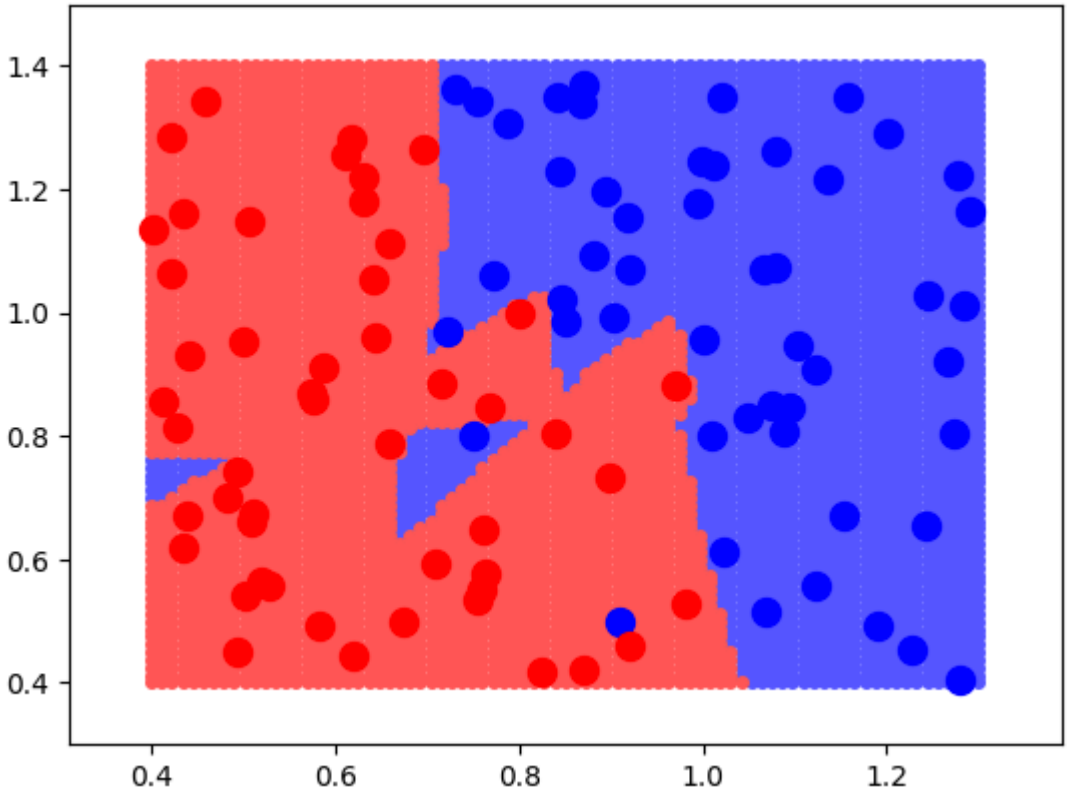
- The model is powerful enough to memorize fine local irregularities, producing a highly adapted boundary.
- In real applications this would risk poor out-of-sample performance, but in this exercise that behavior is explicitly required and demonstrates capacity control in practice.

In [12]:
```
# Creation of a cloud of dots.

# Generate dots_bck test (background) and dots ground truth
dots_bck, dots_gt = hp.p_gen3(100)

# Create batches of 20 elements
inputs, labels = hp.batch_it(20, dots_gt)

# Visualization of the cloud
hp.my_plot(dots_gt, dots_gt)
```



In [13]:
```
class Network_03(nn.Module):

    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(in_features=2, out_features=10, bias=True)
        self.tanh1 = nn.Tanh()
        self.fc2 = nn.Linear(in_features=10, out_features=10, bias=True)
        self.tanh2 = nn.Tanh()
        self.fc3 = nn.Linear(in_features=10, out_features=1, bias=True)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.tanh1(self.fc1(x))
        x = self.tanh2(self.fc2(x))
        x = self.sigmoid(self.fc3(x))
        return x
```

```python
np.random.seed(SEED)
torch.manual_seed(SEED)

# Instanciate the model
model = Network_03()

# Instanciate the optimizer
lr = 0.01
optimizer = optim.Adam(model.parameters(), lr=lr)

# Instanciate the loss function
criterion = nn.BCELoss()

# Simulation
n_epochs = 6000
each = 600
model, _ = train(n_epochs, model, optimizer, criterion, inputs, labels, dots_bck,
          dots_gt, each)
```




```
Acc: 99.0000
```